Lisp — Reference

Lisp here is an interpreter (written in our C, compiled to .NET IL by `cc`): cons cells, interned symbols, numbers, strings, lexical **closures**, recursion, and a small set of special forms and primitives — enough to run a *metacircular evaluator* and even a tiny Lisp-compiler-in-Lisp (Activity 9).

```
lisp prog.lisp        # run a file
lisp                  # interactive REPL (reads s-expressions from stdin)
```

Quick Reference

```
(define x 10)               ; bind a global
(define (f a b) (+ a b))    ; define a function
(lambda (x) (* x x))        ; anonymous function (a closure)
(if c then else)            (cond (t1 e1) (t2 e2) (else e))
(let ((x 1) (y 2)) body)    (begin e1 e2 ...)     (and ...) (or ...)
(quote x)  'x               ; data, not evaluated
car cdr cons list null? pair? eq? equal? not          ; list/atoms
+ - * / = < > ≤ ≥ modulo                              ; arithmetic
print display newline   map append reverse assoc filter length
```

Data types

Numbers (int/double, printed without trailing zeros), **symbols** (interned, so `eq?` is identity), strings (`"..."`), the empty list `()`, cons pairs, and lambda closures. `nil` / `()` is false; everything else is true.

Statements / Commands

Lisp is expression-only. Special forms: `quote`, `if`, `cond`, `lambda`, `define`, `let`, `begin`, `and`, `or`, `set!`. Everything else is a function application `(fn arg …)`, evaluated by pushing the evaluated arguments and applying.

Functions

`(define (name params…) body…)` or `(define name (lambda …))`. Functions are first-class closures: they capture their defining environment, recurse, and can be returned from other functions (Activity 5). A startup **prelude** (in Lisp) defines `map`, `append`, `reverse`, `assoc`, `filter`, `length`, `equal?`, `member`, etc.

Input / Output

`(print x)` writes a value and a newline; `(display x)` writes without a newline; `(newline)` writes a line break. Values print in standard notation: lists as `(a b c)`, the empty list as `()`.

Graphics

None. Activity 8 below is **text art** drawn with `display` / `newline`.

Notes

- The interpreter runs on .NET IL (it is itself compiled by `cc`); the **programs** it runs are interpreted.
- Symbols are interned, so `eq?` on symbols is pointer identity; `equal?` compares structure.
- `eval` and `apply` are exposed as primitives — which is what lets a Lisp program evaluate Lisp (Activity 9).

Subset boundaries

A clean core, not Common Lisp / Scheme in full. Not included: macros, tail-call optimization (deep non-tail recursion can overflow), vectors/hash-tables, string manipulation beyond literals (no `string-append`), `call/cc`, exceptions, and the full numeric tower (int + double only). Lists are the data structure.

Tutorial

Every example was run with `lisp`; the output shown is real.

1. Your first program

```
(print "Hello, Lisp!")
```

```
Hello, Lisp!
```

A program is a sequence of expressions, evaluated top to bottom; `print` shows a value.

2. Variables and data types

```
(print (+ 40 2))
(define greeting "hi")
(print greeting)
(if (> 5 3) (print (quote yes)) (print (quote no)))
```

```
42
hi
yes
```

`define` binds a global. Numbers, the string `"hi"`, and the symbol `yes` (quoted, so it is data) are all values.

3. Flow control

```
(print (cond ((= 1 2) (quote a)) ((= 2 2) (quote b)) (else (quote c))))
```

```
b
```

`if` chooses between two expressions; `cond` walks clauses until a test is true (`else` always matches). `and` / `or` short-circuit.

4. Arrays

Lisp's data structure is the **list**, not the array — and the prelude gives the usual operations:

```
(print (map (lambda (x) (* x x)) (list 1 2 3 4 5)))
(print (append (list 1 2) (list 3 4)))
(print (reverse (list 1 2 3)))
(print (filter (lambda (x) (> x 2)) (list 1 2 3 4 5)))
```

```
(1 4 9 16 25)
(1 2 3 4)
(3 2 1)
(3 4 5)
```

`map` / `filter` take functions as arguments — lists plus higher-order functions cover what arrays do elsewhere.

5. Subroutines and functions

```
(define (fact n) (if (= n 0) 1 (* n (fact (- n 1)))))
(print (fact 10))
(define (fib n) (if (< n 2) n (+ (fib (- n 1)) (fib (- n 2)))))
(print (fib 15))
(define (make-adder n) (lambda (x) (+ x n)))
(define add5 (make-adder 5))
(print (add5 10))
```

```
3628800
610
15
```

`fact` / `fib` recurse. `make-adder` returns a **closure** that captures `n`, so `add5` adds 5 — first-class functions in action.

6. Memory management

Lisp's model is cons cells and a garbage collector:

- `cons` builds a pair; `car` / `cdr` take it apart; a list is pairs ending in `()`.
- **Symbols are interned** — every `'foo` is the same object, so `eq?` is a pointer compare.
- **Environments** (the bindings `define` / `let` create) are themselves lists of pairs; closures hold a reference to theirs.
- Nothing is freed by hand — discarded cells are reclaimed by .NET's GC.

```
(print (assoc (quote b) (list (list (quote a) 1) (list (quote b) 2))))
(let ((x 3) (y 4)) (print (+ x y)))
```

```
(b 2)
7
```

`assoc` searches an association list (pairs); `let` makes a local environment.

7. Strings and a text layout

Strings are literals printed with `print` / `display`; structured text is usually built from symbols and lists. To lay out text you recurse with `display`:

```
(define (repeat-char c n)
  (if (= n 0) (newline) (begin (display c) (repeat-char c (- n 1)))))
(repeat-char (quote =) 10)
```

```
==========
```

(Strings have no `string-append` in this subset — see *Subset boundaries*; symbols and lists carry structured text.)

8. Drawing a picture

No graphics — **text art** via recursion:

```
(define (row n) (if (= n 0) (newline) (begin (display (quote *)) (row (- n 1)))))
(define (tri n i) (if (> i n) (quote done) (begin (row i) (tri n (+ i 1)))))
(tri 4 1)
```

```
*
**
***
****
```

9. Where to go next

Because `eval` / `apply` / `cons` are primitives, Lisp can implement Lisp. Two showcase programs ship with the interpreter:

- **A metacircular evaluator** (`meta.lisp`) — `eval` / `apply` written in Lisp, evaluating expressions including a **Y-combinator factorial** (recursion with no `define`):

`25 yes (7 7) 23 720`

- **A Lisp compiler written in Lisp** (`compiler.lisp`) — compiles an arithmetic/ `if` / `let` sublanguage to stack **bytecode** and runs it on a VM:

`bytecode for p3: ((PUSH 2) (PUSH 3) (ADD) (PUSH 10) (PUSH 4) (SUB) (MUL)) run p3 = 30 run p1 (x=3) = 6 run p2 (x=10) = 110`

From here, read those two programs — they are the deep end of "Lisp in Lisp."